

TP1 - EXCEPTIONS ET COLLECTIONS

Toutes les classes doivent être définies dans des sous packages de **sn.uasz.m1.tp1**.

Exercice 1

Analyser le bout de code ci-dessous et indiquer sa sortie.

```
1 import java.util.*;
2
3 class Exception1 extends Exception {
4     Exception1(String s) { super(s); }
5 }
6 class Exception2 extends Exception {
7     Exception2(String s) { super(s); }
8 }
9 class Exception3 extends Exception1 {
10     Exception3(String s) { super(s); }
11 }
12 class TestException {
13     static void testLevee(int i) throws Exception {
14         switch (i) {
15             case 1: throw new Exception1("exception de type Exception1");
16             case 2: throw new Exception2("exception de type Exception2");
17             case 3: throw new Exception3("exception de type Exception3");
18             default: throw new Exception("exception de type Exception");
19         }
20     }
21     public static void main(String [] args){
22         for (int i = 1; i < 5; i++) {
23             try { testLevee(i); }
24             catch (Exception3 e){System.out.println("Capture d' "+e.getMessage());}
25             catch (Exception1 e){System.out.println("Capture d' "+e.getMessage());}
26             catch (Exception e){System.out.println("Capture d' "+e.getMessage());}
27             finally System.out.println("Exécution du bloc Finally de main");}
28         }
29     }
30 }
```

Exercice 2

On considère une classe Java nommée `Operation` ayant pour attributs deux opérandes de type entier.

1. Créer la classe `Operation` comportant :

- un constructeur par défaut et un constructeur d'initialisation ;
- une méthode `quotient()` qui renvoie le quotient de la première opérande par la deuxième si cette dernière n'est pas nulle. Sinon elle lève une exception de type `OperationException` (classe à définir) ;
- une méthode `racineDeLaSomme()` qui renvoie la racine carrée de la somme des deux opérandes si cette somme est positive ou nulle. Sinon elle lève une exception de type `OperationException` ;
- une méthode `estDivisible()` qui renvoie `true` si la première opérande est divisible par la deuxième, `false` sinon. Dans le cas où une des opérandes est négative ou nulle, elle lève une exception de type `OperationException`.

2. Créer une classe `TestOperation` contenant une méthode `public static void main (String [] args)` pour tester ces différentes opérations.

Exercice 3

On souhaite réaliser un programme de gestion des employés d'une société. Un employé sera représenté par son matricule, son nom, son salaire et sa date d'embauche.

1. Définir la classe **Employe** avec un constructeur paramétré, des getters et setters et une méthode `toString()`.
2. Créer une classe **GestionEmploye** qui fournit un jeu d'essai dans sa méthode `main()`. Cette classe doit permettre de :
 - enregistrer un employé de la société ;
 - déterminer la liste des employés embauchés à une date donnée ;
 - déterminer la liste des employés embauchés avant une date donnée ;
 - calculer le nombre d'employés embauchés avant une date donnée ;
 - déterminer la liste des employés embauchés avant une date donnée et gagnant un salaire supérieur à un montant ;
 - déterminer l'employé le mieux payé.

Exercice 4

On souhaite développer un programme Java pour gérer les membres du club informatique de l'UASZ. Un membre est une personne représentée par un identifiant, son nom, sa profession, son sexe (homme ou femme) et son numéro de téléphone.

1. Créer la classe **Membre** avec les méthodes suivantes :
 - un constructeur par défaut et un constructeur d'initialisation ;
 - les getters et setters de ses attributs ;
 - une méthode `toString()` de cette classe ;
 - une méthode `equals()` permettant de vérifier que deux membres sont identiques (même identifiant).
2. Créer la classe **Club** avec un seul attribut, la liste de ses membres représentée par une collection de type **Map** (e.g **HashMap**) dont la clé est l'identifiant du membre. Cette classe comporte les méthodes suivantes :
 - une méthode `ajouter(Membre membre)` permettant d'ajouter un nouveau membre au club. Cette méthode lève une exception de type **IdentifiantExistantException** (classe à définir) si l'identifiant est utilisé pour un autre membre ;
 - une méthode `modifier(String identifiant)` permettant de mettre à jour les informations d'un membre. Cette méthode lève une exception de type **InexistantMembreException** (classe à définir) si l'identifiant spécifié n'existe pas ;
 - une méthode `supprimer(String identifiant)` qui permet de supprimer un membre. Cette méthode lève une exception de type **InexistantMembreException** si l'identifiant spécifié n'existe pas ;
 - une méthode `afficher()` permettant d'afficher l'ensemble des membres du club.
3. Créer une classe **GestionClub** contenant une méthode `public static void main (String [] args)` pour tester ces différentes méthodes.

Exercice 5

On désire réaliser une application Java permettant de gérer un parking automobile. Les voitures seront stationnées sur des places numérotées à partir de 1. La capacité (nombre de places) du parking est fixe. Une voiture est caractérisée par un numéro d'immatriculation, sa marque, son kilométrage, le type de carburant et sa couleur.

1. Créer la classe **Voiture** avec les méthodes suivantes
 - un constructeur avec des paramètres
 - les getters et setters de ses attributs ;
 - une méthode **toString()** ;
 - une méthode **equals()** permettant de vérifier l'égalité de deux voitures (mêmes numéros et marques).
2. Créer la classe **Parking** avec deux attributs : sa capacité et la liste des voitures stationnées dans le parking, représentée par une collection de type **Map** ;
3. Définir un constructeur avec des paramètres, les getters et setters des attributs ;
4. Définir une méthode **public void garer(Voiture v, int place)** qui gare la voiture à la place spécifiée si elle est libre. Cette méthode lève une exception de type **PlaceInexistanteException** (classe à définir) si le numéro de place n'existe pas et une exception de type **PlaceOccupeeException** (classe à définir) si la place est occupée ;
5. Définir une méthode **retirer(int place)** qui retire du parking la voiture à cette place. Cette méthode lève une exception de type **PlaceInexistanteException** si le numéro de place n'existe pas et une exception de type **PlaceNonOccupeeException** (classe à définir) si la place est libre ;
6. Écrire une méthode **afficher()** permettant d'afficher l'état du parking : pour chaque place, le numéro, ainsi que les caractéristiques de la voiture qui l'occupe si elle existe.
7. Écrire une méthode **public int placeOccupee(int numero)** qui prend en paramètre un numéro d'immatriculation et retourne le numéro de place occupée par la voiture ou -1 si elle n'existe pas.
8. Créer une classe **GestionParking** contenant une méthode **public static void main (String [] args)** pour tester ces différentes méthodes.

Exercice 6

Dans cet exercice, on veut écrire un programme Java permettant de traiter des textes. Un texte est une chaîne de caractères. Les mots sont des séquences de caractères délimités par des espaces, virgule, point, etc. Pour décomposer un texte en mots, on peut utiliser la classe **StringTokenizer** qui possède :

- un constructeur qui prend deux paramètres : le texte à découper et l'ensemble des séparateurs ;
- une méthode **nextToken()** qui retourne le mot courant ;
- une méthode **hasMoreTokens()** qui retourne **true** s'il en reste d'autres tokens, **false** sinon.

```
1 String texte = "Bonjour tout le monde. Bienvenue au cours de programmation Java.";
2 StringTokenizer tokenizer = new StringTokenizer(texte, " .;");
3 // les séparateurs sont " ", "." et ";";
4 while(tokenizer.hasMoreTokens())
5     System.out.println(tokenizer.nextToken()); // affiche les tokens ou mots
```

Listing 1 – Exemple

1. Créer la classe `ManipTexte` avec un seul attribut nommé `texte` de type `String`, définir son constructeur avec paramètre, ses getters et setters;
2. Écrire une méthode `public HashMap<String, Integer> compte()` permettant de calculer la fréquence de chaque mot du texte et de la stocker dans un dictionnaire.
3. Écrire une méthode `plusFrequent()` qui retourne le mot le plus fréquent du texte avec son nombre d'occurrences.
4. Écrire une méthode `public void afficherMots()` permettant d'afficher les mots avec leurs fréquences dans le texte.
5. Écrire une méthode `public void afficherMotsEnOrdre()` pour afficher les mots dans l'ordre décroissant de leurs fréquences.
6. Créer une classe `TestManipTexte` avec une méthode `public static void main (String [] args)` pour tester ces différentes méthodes.

Exercice 7

On considère l'interface `Pile` suivante :

```
1 package exercice2;
2 public interface Pile {
3     public boolean isEmpty() ;
4     public void push(Object obj) ;
5     public Object pop() throws EmptyPileException;
6 }
```

L'interface `Pile` décrit l'interface standard d'une pile d'objets :

- la méthode `boolean isEmpty()` indique si la pile est vide ou pas ;
- la méthode `void push(Object obj)` empile l'objet `obj` dans la pile ;
- la méthode `Object pop()` renvoie le dernier objet empilé et dépile la pile.

La méthode `pop()` ne peut pas retourner de valeur quand la pile est vide. Dans ce cas, une exception de type `EmptyPileException` doit être levée.

1. Écrire la classe `EmptyPileException` en étendant la classe `Exception`.
2. Écrire une classe `PileList` qui implémente l'interface `Pile` et réalise une pile du type *Last In, First Out*. Pensez à lever une exception quand c'est nécessaire. Comme structure interne de votre pile, vous utiliserez un attribut de type `ArrayList`.
3. Écrire une classe `TestPile` qui crée une instance de la classe `PileList`, empile trois chaînes de caractères et essaie de dépiler la pile et d'afficher la dernière valeur. N'oubliez pas de capturer les exceptions qui peuvent être levées.